Department Research Colloquium 2026

RAP twin: A Digital Twin and Planning Sandbox for Distributed Systems

- Riaan Masoud Attar (first author)
- Avanti Nitin Patil
- Preet Prashant Boralkar
- Faculty Mentor: Prof. K. K. Deshmukh

- Department of Artificial Intelligence and Data Science
- K. K. Wagh Institute of Engineering Education and Research, Nashik

- Group 01 | 30 October 2026

Abstract

• Evaluating job-scheduling policies for edge, multi-cloud and hybrid fabrics on production hardware is risky and costly. RAP twin is a digital-twin and planning sandbox that mirrors a 100-node heterogeneous fabric from YAML descriptors and live telemetry. It plans multi-stage jobs with seven pluggable strategies, injects deterministic faults, heals itself, and scores every plan for latency, energy, cost and carbon. On the reference fabric, five strategies place identically in dry-run, Resilient predicts 19% higher latency and about 40× the cost in exchange for redundancy, and RL-Markov leaves 3 of 11 jobs infeasible under contention. A chaos-as-CI gate and a 1,000-trial Monte-Carlo sweep make these trade-offs reproducible. The study reports its measured results and known limits openly.

Keywords: digital twin • distributed systems • task scheduling • chaos engineering • Monte-Carlo simulation • edge computing

Introduction and Motivation

Mixed fabrics

• Servers, workstations, single-board computers, phones and HPC nodes with different CPUs, GPUs, links and reliability

Deadline pipelines

• Jobs are multi-stage pipelines with SLAs; placement balances latency, energy, cost, carbon and risk

Risk on production

• Trying a new scheduler live is dangerous: link outages and thermal throttling cause real failures

Digital twins

• A synchronized virtual copy lets policies be tried safely [1]–[3]

RAP twin

• One sandbox uniting planning, fault injection, self-healing and benchmarking

Problem Definition and Research Gap

Problem

- Choose, for every stage of a job, a node that meets resource and format limits while balancing latency, energy, cost, carbon and risk
- Keep the job alive when nodes or links fail

Research gap

- Twin offloading studies focus on learning algorithms (multi-agent / federated deep RL) [4], [5], not a reusable sandbox for comparing planners
- Chaos tools target software and containers, largely ignoring physical-layer faults [6], [7]
- No single sandbox found combining live twin, several planners, chaos testing, cost/carbon accounting and a regression gate
- Aim: report measured trade-offs and defects openly

Objectives and Scope

Objectives

1. Multi-strategy planning: Greedy, Cheapest-Energy, Cheapest-Cost, Greenest, Bandit, Resilient, RL-Markov
2. Real-time state sync: YAML + live telemetry (REST / MQTT)
3. Predictive analytics and self-healing
4. Deterministic chaos engineering and a chaos-as-CI gate
5. Large-scale Monte-Carlo benchmarking
6. What-if capacity and blast-radius analysis
7. Interoperability: Kubernetes import, Azure DTDL / CRD export

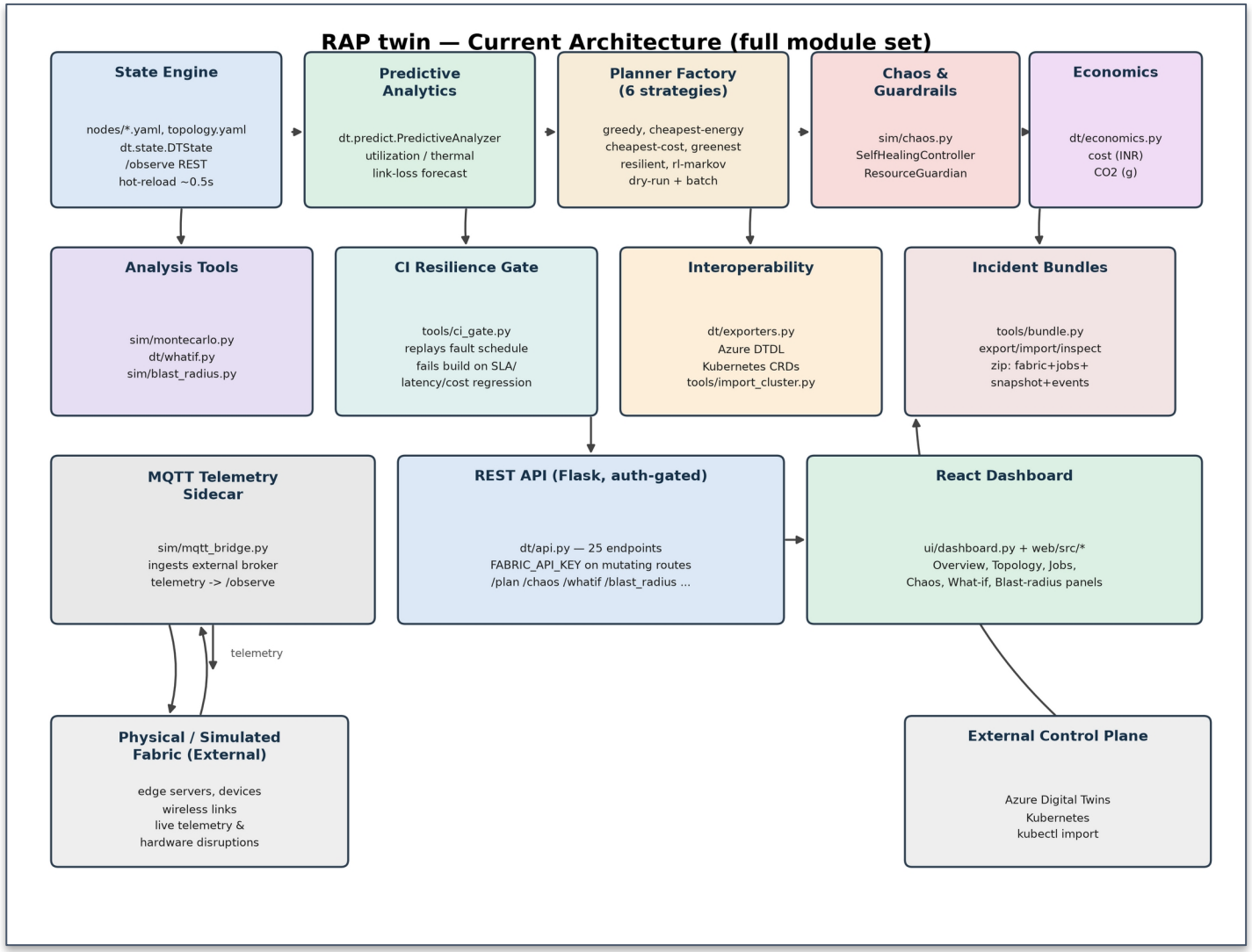
In scope

- Simulated policy evaluation
- Dashboard, REST API, benchmarks

Out of scope

- Running real workloads
- Production scheduler
- Training deep RL agents
- Live 5G/6G test-bed

System Architecture



RAP twin layered architecture

Layers

- Presentation: REST/SSE API, React dashboard
- Orchestration: planner factory, chaos runner, self-heal, guardian
- Decision: planners, cost model, economics, predictor
- State: DTState, event bus, validators
- Data: YAML descriptors, topology, overrides, jobs

Proposed Methodology

Ingest state → Predict → Plan → Inject faults → Self-heal → Benchmark

State and prediction

- DTState merges YAML + telemetry; hot reload ≈ 0.5 s
- Predictor: utilization, thermal derating, link loss

Planning engine

- Pluggable DAG planners; dry-run and commit modes
- Greedy family, Bandit, Resilient (fallbacks, spread)
- RL-Markov: Tchebycheff over nine cost terms

Chaos and healing

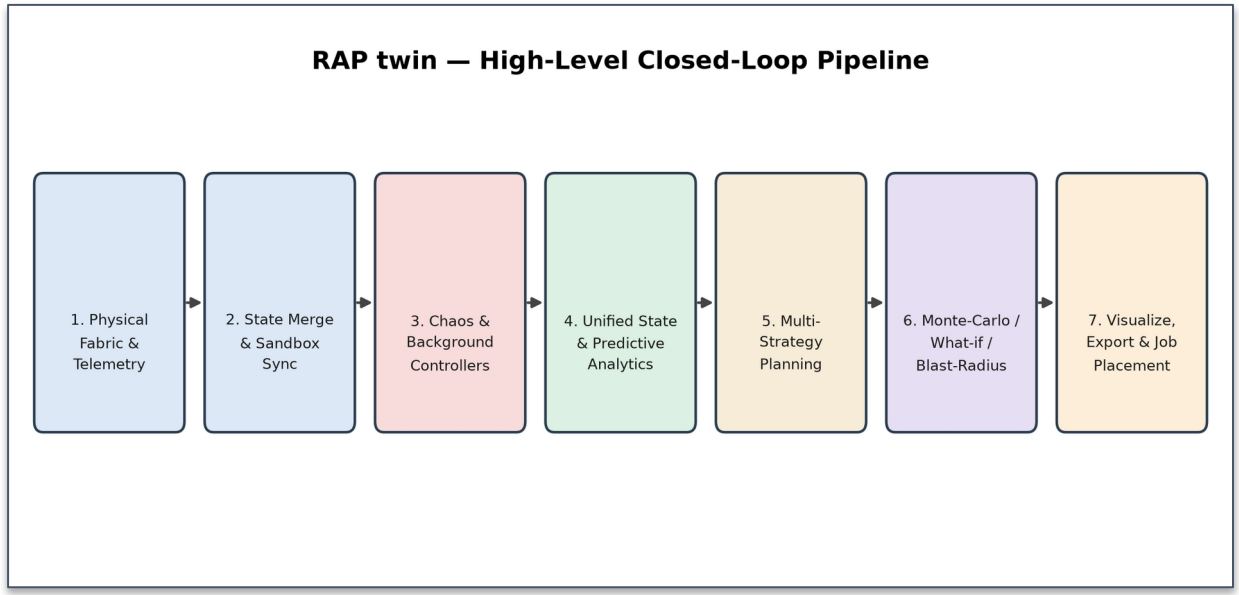
14 fault kinds: link cut, noise, thermal derate, zone blackout

- SelfHealingController and ResourceGuardian

Tools and stack

- Monte-Carlo, CI gate, blast radius, what-if, bundles
- Flask API + React/TypeScript dashboard (SSE)

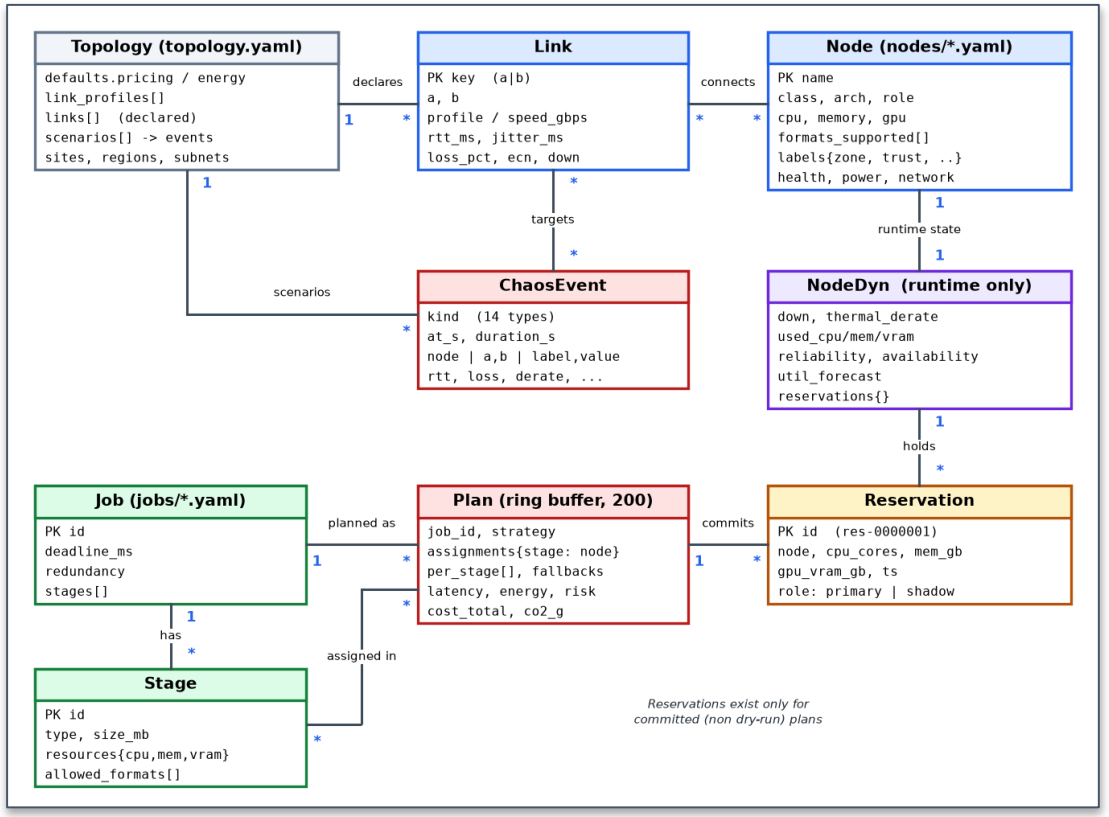
Pipeline and Data Model



Closed-loop pipeline

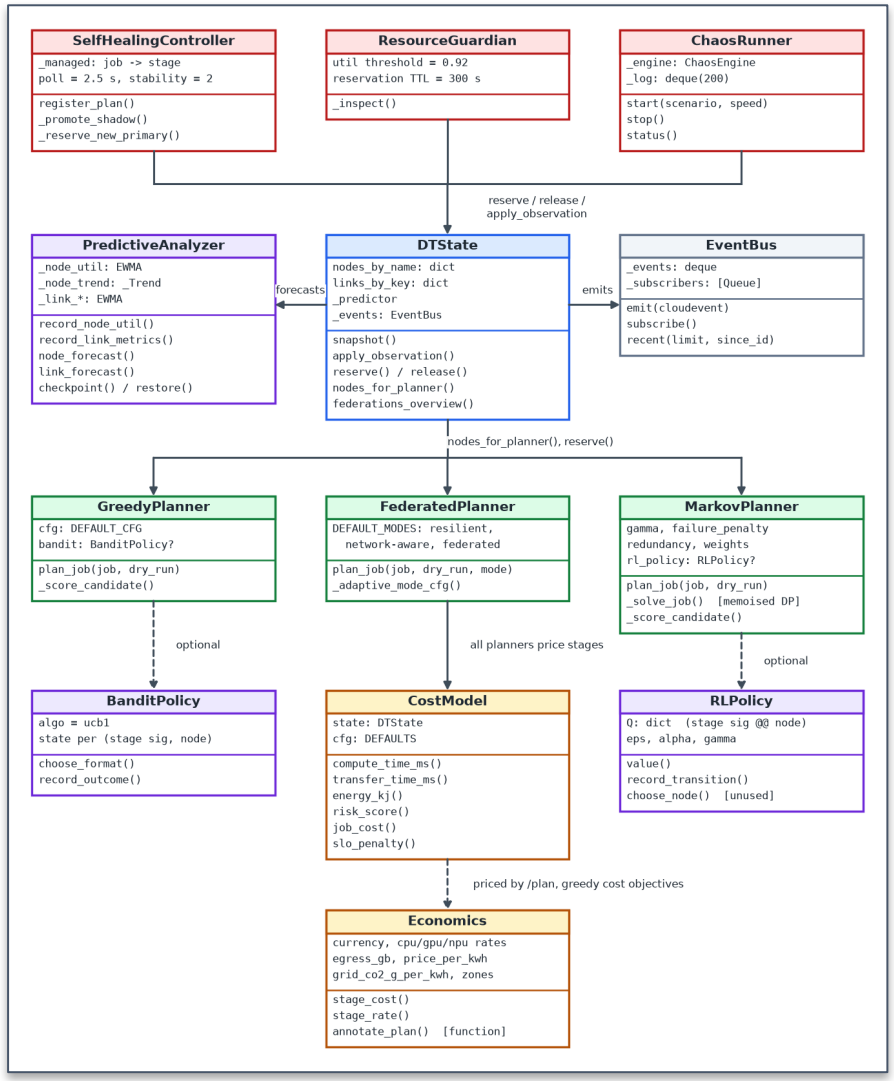
Three-layer node state

- base: static YAML descriptor + derived caps
- dyn: down, derate, used CPU/mem/VRAM, reliability, reservations
- effective: free capacity = cores x (1 - derate) - used

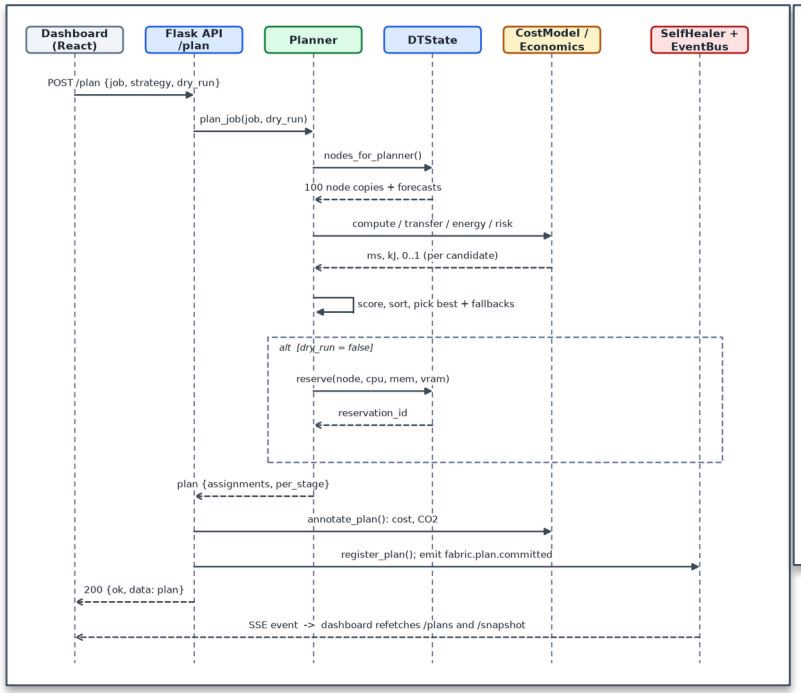


Entity-relationship diagram

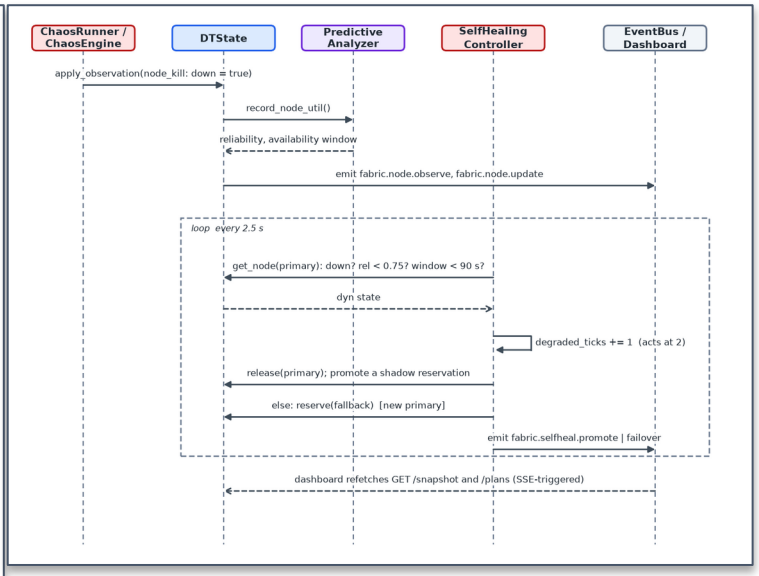
Class and Sequence Design (UML)



Class diagram (dt/ core)



Planning a job



Fault and self-healing

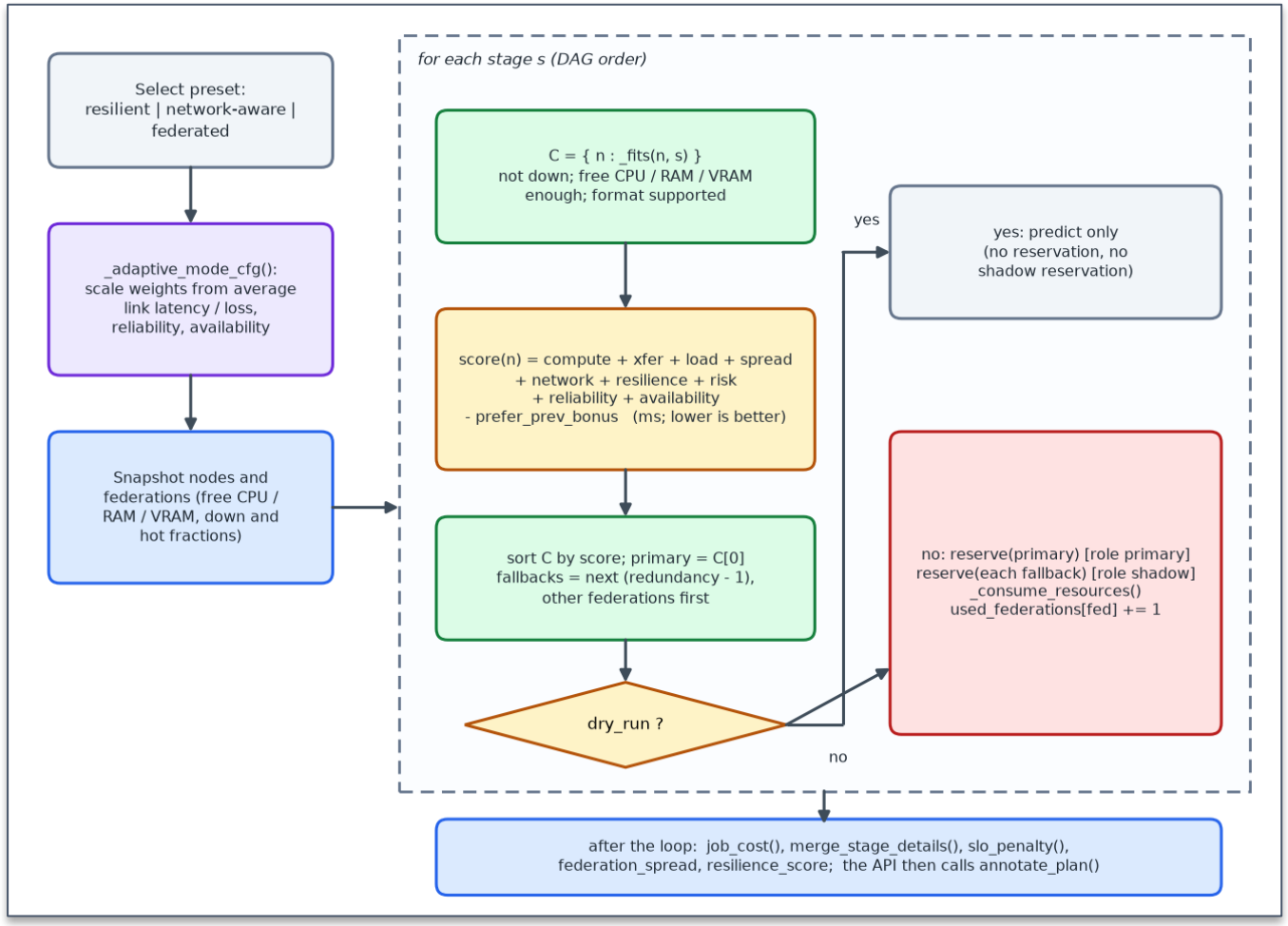
Strategies and Cost Model

Strategy	Class	Objective
greedy	GreedyPlanner	compute + transfer + 10*risk
cheapest-energy	GreedyPlanner	+ 0.1 ms per kJ
cheapest-cost	GreedyPlanner	+ 25 ms per Rs/hour
greenest	GreedyPlanner	+ 25 ms per gCO2/hour
bandit	Greedy + UCB1	learns execution format
resilient	FederatedPlanner	additive ms score, fallbacks
rl-markov	MarkovPlanner	MDP + Tchebycheff, gamma 0.92

Cost model (per stage)

- Compute: W = 2*size + 120*cores; t = W/(units/10*accel)*(1+0.35*util) + 6000*(1-reliability)
- Transfer: 8*size/(speed*850*(1-loss)) + RTT + jitter
- Energy: P*(0.12 + 0.88*u^0.85)*t
- Risk: weighted blend of trust, wear, crashes, derate, loss, reliability, availability
- SLO penalty: ((L/D)^1.2 - 1)/0.002
- Cost: rental + electricity + egress; CO2 = kWh x 720 g

Resilient Planner: Algorithm



Flowchart of the resilient planner

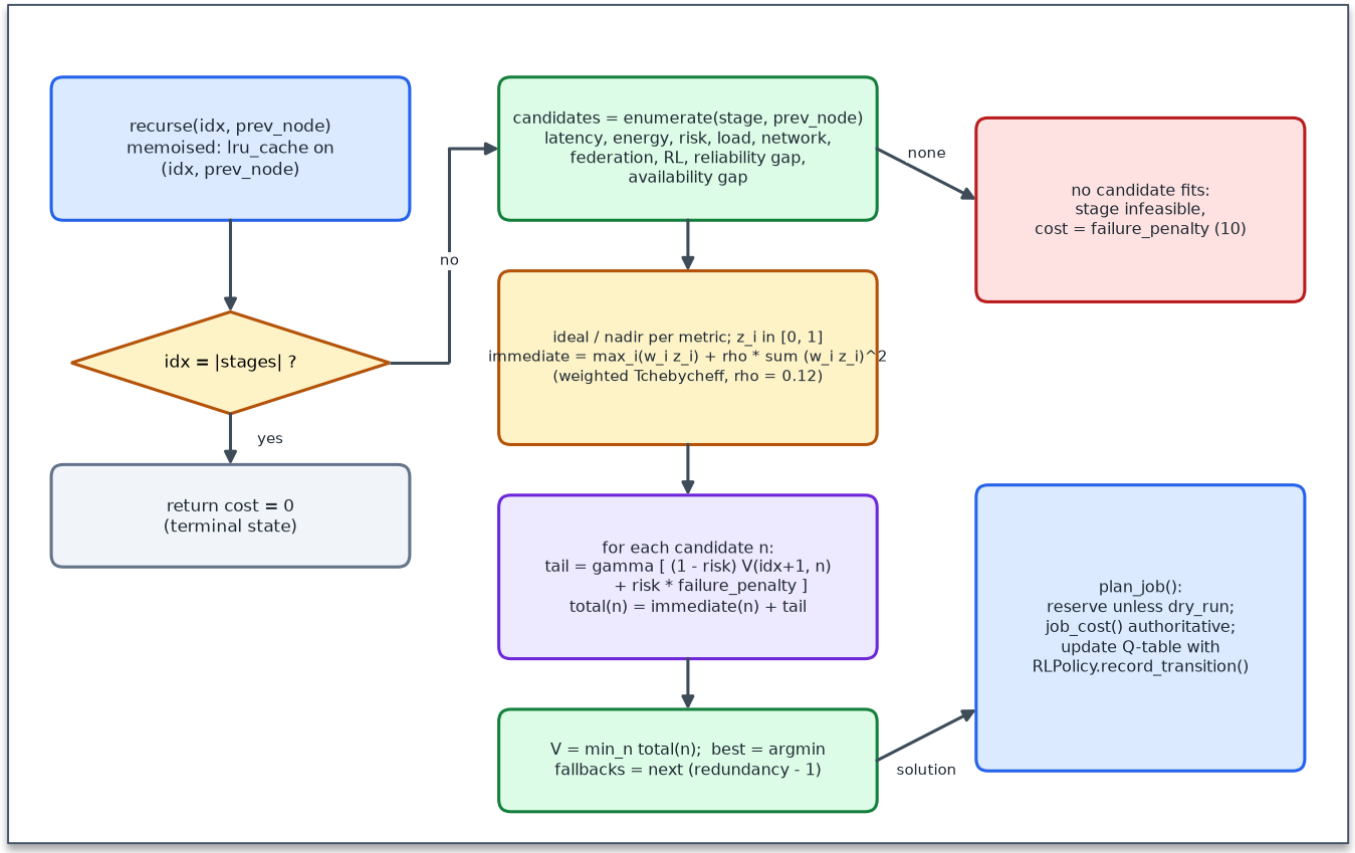
Additive score (ms, lower is better)

- compute + xfer + load + spread + network + resilience + risk + reliability + availability - prev-bonus

Preset	Redund.	Load	Spread	Net
resilient	2	380	210	240
network-aware	1	260	140	300
federated	3	360	260	230

- Fallbacks prefer other federations; committed plans reserve shadow nodes that the self-healer can promote instantly.

RL-Markov Planner: MDP



Memoised dynamic-programming recursion

- Complexity O(m* |N|^2): about 35,000 cost evaluations per job, hence 1.6 s planning.

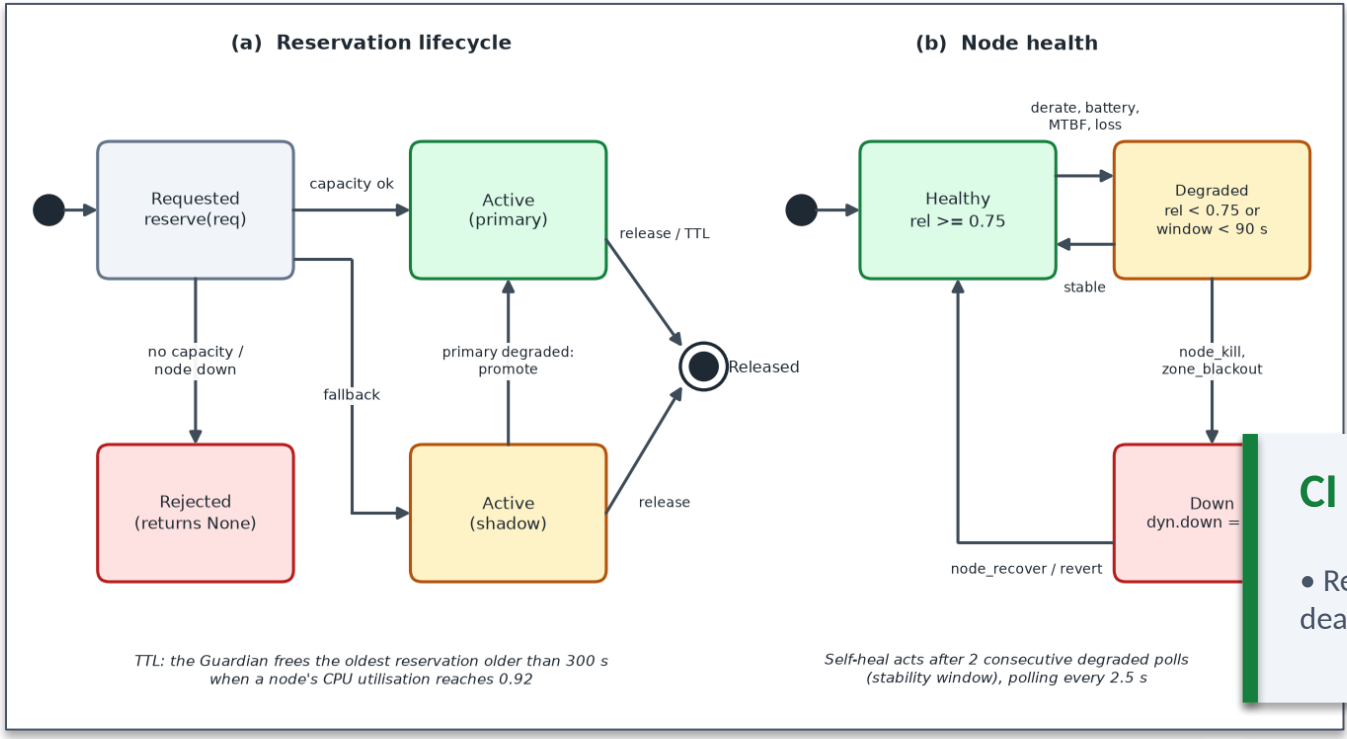
Caveat: stages are scored against an unmodified snapshot, so they can be stacked on one node.

Bellman recursion

- V(i,p) = min_n [c_i(n) + gamma((1-rho_n) V(i+1,n) + rho_n F)]
- gamma = 0.92, failure penalty F = 10

Criterion	Weight
latency	0.34
risk	0.16
energy	0.14
load	0.10
network	0.08
federation	0.06
RL, reliability, availability	0.04 each

Chaos Engine, Self-Healing and CI Gate



Family	Event kinds
Link (4)	degrade, loss_spike, down, up
Node (7)	kill, recover, power_cap, thermal_derate, clock_skew, packet_dup, packet_reorder
Group (3)	zone_blackout, zone_recover, federation_partition

CI gate

- Replays faults; fails build on SLA/p95/cost regression vs baseline (seed 7, deadline scale 0.28)

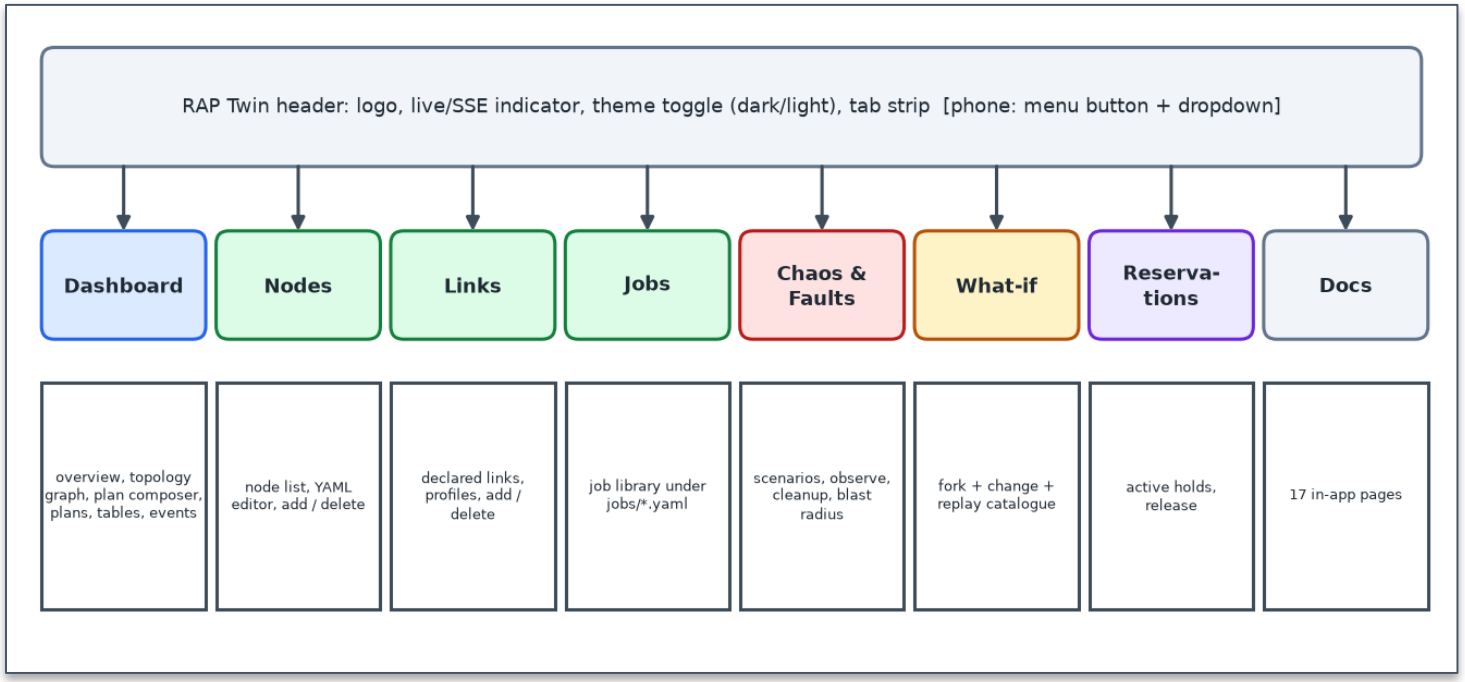
Self-healing

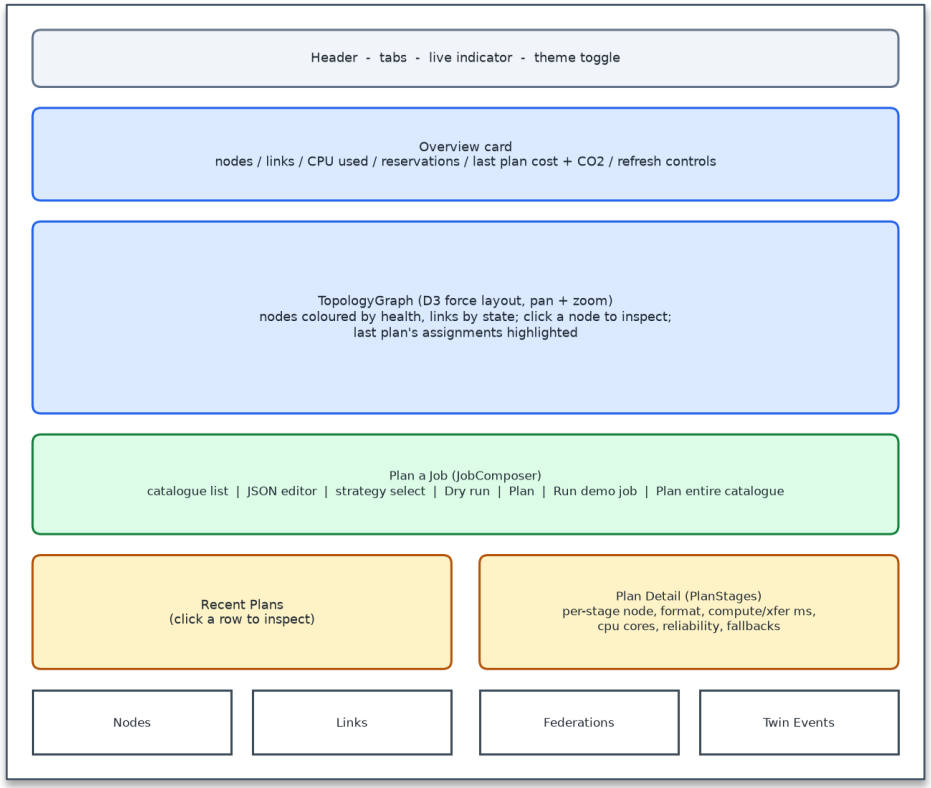
- Poll 2.5 s; degraded if down, reliability < 0.75 or window < 90 s
- Acts after 2 polls: promote shadow, else fail over; Guardian frees stale reservations

Analysis tools

- Blast radius: BFS minimal breaking fault sets
- What-if: fork twin, apply change, diff
- Monte-Carlo: seeded trials, mean/p95/p99

Dashboard (UI)





Dashboard tab composition

Features

- Live updates via SSE; dark/light theme; phone-friendly menu
- Plan composer with dry-run, 17-page in-app docs, per-stage plan detail
- Chaos, blast radius, what-if, reservations, incident bundle download

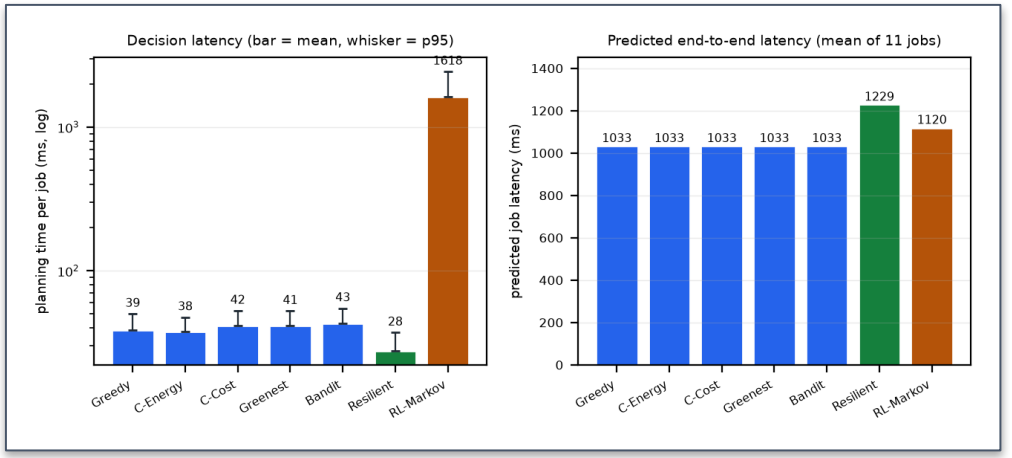
Experimental Setup

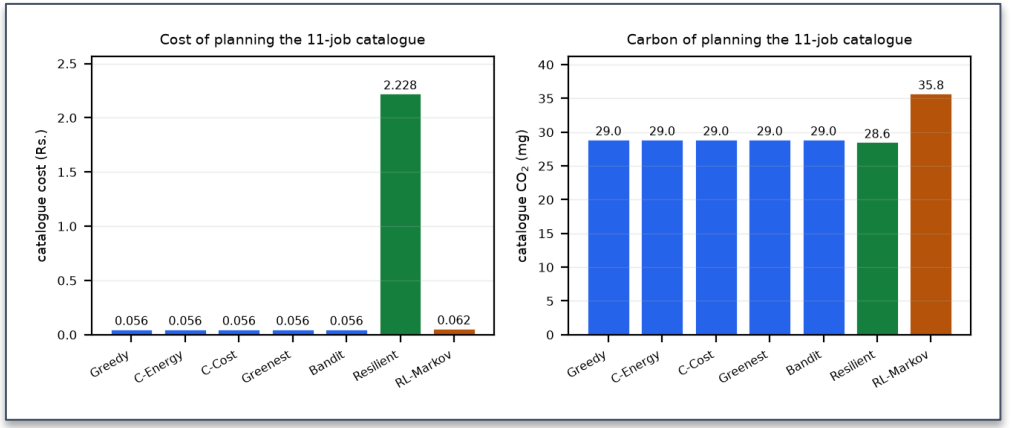
100	2,022	10,508 GB	51	11 / 36
nodes, 7 classes, 7 zones	CPU cores	RAM	GPU nodes	jobs / stages

- Fabric: links from 1 Gbps WAN to 400 Gbps InfiniBand
- Workload: vision, NLP, analytics, audio, tabular, HPC and mixed-accelerator jobs, each with a deadline
- Experiments: (a) dry-run comparison (b) committed planning with contention (c) chaos gate under zone events (d) 1,000-trial Monte-Carlo (e) what-if and blast radius
- Benchmark table: commit 2e7962d, seed 7, first 6 jobs, deadline scale 0.28
- All results come from the project's own tools and are reproducible

Results: Dry-Run Strategy Comparison

Strategy	Plan ms	Latency ms	Energy kJ	Cost Rs	CO2 mg
Greedy	38.7	1,033	0.1406	0.0563	29.0
Cheapest-Energy	37.6	1,033	0.1406	0.0563	29.0
Cheapest-Cost	41.5	1,033	0.1406	0.0563	29.0
Greenest	41.4	1,033	0.1406	0.0563	29.0
Bandit	43.1	1,033	0.1406	0.0563	29.0
Resilient	27.7	1,229	0.1417	2.2284	28.6
RL-Markov	1,617.8	1,120	0.1782	0.0622	35.8

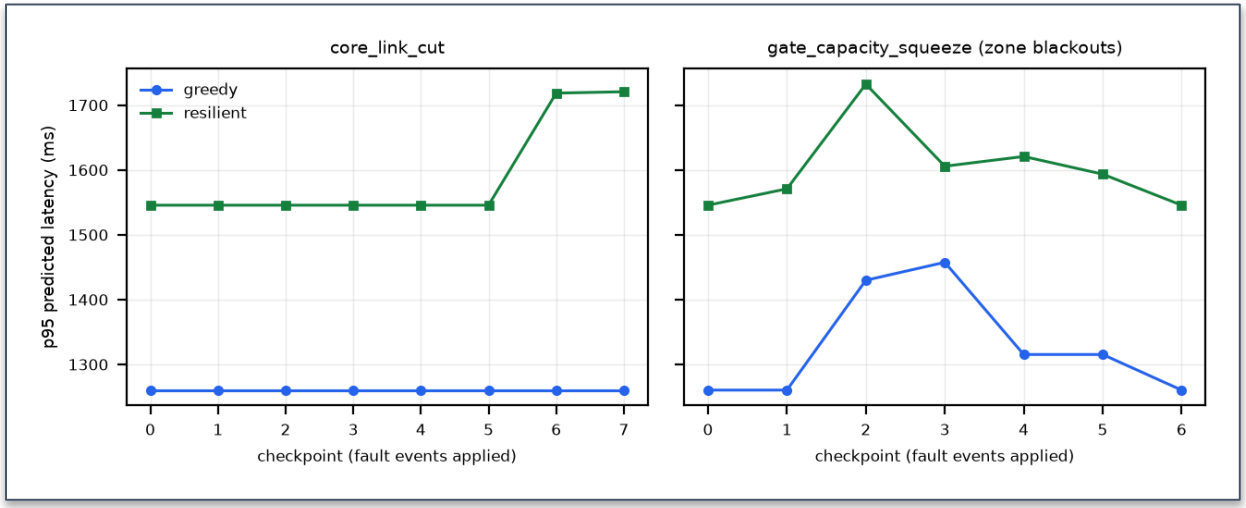




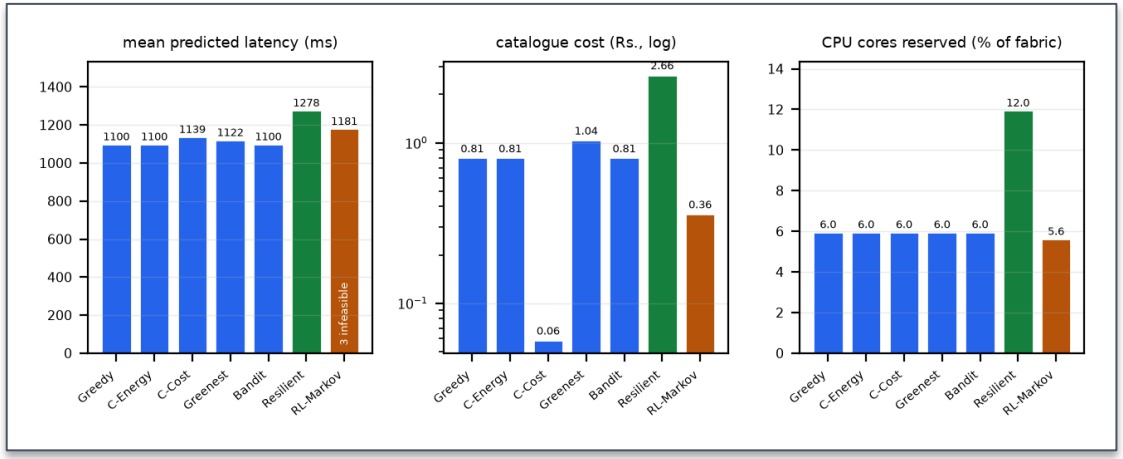
- Five strategies place all 36 stages on hpc-080
- Resilient: 97% of cost is cross-zone egress (2.9 zones/job)
- Greedy planning 38.7 ms vs SRS target 5 ms: about 60% is node-table copying

Results: Contention, Gate and What-If

Strategy	Infeas.	Mean ms	Cost Rs	Reserv.
Greedy	0	1,100	0.808	36
Cheapest-Cost	0	1,139	0.059	36
Greenest	0	1,122	1.042	36
Resilient	0	1,278	2.659	72
RL-Markov	3	1,181	0.362	33



p95 latency per gate checkpoint



What-if change	Deadlines	p95 ms	Cost Rs
+2 x srv-015	11 to 11	1,409 to 1,409	0.808 to 0.808
- hpc-080	11 to 11	1,409 to 1,466	0.808 to 1.276
- 3 HPC nodes	11 to 11	1,409 to 1,433	0.808 to 0.526

- Blast radius: 16 faults, 62 placements; both jobs survive. Monte-Carlo, 1,000 trials: 516 ms mean, 632 ms p99, 100% SLO.

Testing and Known Limitations

Suite	Tests
API + auth + management	34
CI gate	19
Economics	11
Blast radius	13
What-if	18
Bundle / import / MQTT / launcher	61
Benchmarks	10
Total (all pass)	166

Limitation	Effect
Link p95 forecast uses accumulator	about 7e12 ms on registered links
Trend with epoch-ms timestamps	30 s look-ahead inert
WASM penalty clamped	no effect
MDP ignores in-job capacity	3/11 jobs infeasible
core_link_cut targets absent nodes	no effect on plans
Schemas / validators not wired	25/100 nodes fail
Predictions, not executions	single seed

166 tests pass in 8.6 s; the gate is deterministic (byte-identical reruns).

Conclusion and Future Scope

Conclusion

- Working, deployed sandbox with 166 passing tests
- No strategy wins every metric
- Rate-based carbon/cost objectives are proxies
- Gate, scripts and honest limits are results themselves

Limitations

- Latencies are model predictions, not executed workloads
- Single seed
- Over-provisioned fabric

Future scope

- Algorithm recommendation: rank strategies per job
- Fix RL-Markov capacity accounting and known defects; re-baseline gate
- Validate against real containers; real RL loop; 5G/6G test-bed

References and Acknowledgment

[1] Q. Qi et al., "A review of digital twins in smart industries," IEEE Access, vol. 7, pp. 14232–14252, 2019.

[2] A. Fuller et al., "Digital Twin: Enabling technologies, challenges and open research," IEEE Access, vol. 8, pp. 108952–108971, 2020.

[3] Z. Lv et al., "Digital twins: A survey on enabling technologies, challenges, trends, and future prospects," IEEE Commun. Surveys Tuts., vol. 24, no. 1, pp. 54–87, 2022.

[4] C. Xu et al., "Digital twin-driven collaborative scheduling for heterogeneous task and edge-end resource via multi-agent deep reinforcement learning," IEEE JSAC, vol. 41, no. 10, pp. 3056–3069, 2023.

[5] Y. Dai et al., "Federated deep reinforcement learning for task offloading in digital twin edge networks," IEEE TNSE, vol. 11, no. 3, pp. 2849–2861, 2024.

[6] E. Esen, A. Akbulut and C. Catal, "Chaos experiments in microservice architectures: A systematic literature review," Comput. Stand. Interfaces, vol. 97, p. 104116, 2026.

[7] F. Poltronieri, M. Tortonesi and C. Stefanelli, "ChaosTwin: A chaos engineering and digital twin approach for the design of resilient IT services," in Proc. 17th CNSM, 2021.

• Acknowledgment: We thank our guide Prof. K. K. Deshmukh, the Head of Department Prof. Dr. D. V. Medhane, and the Department of AI and DS, KKWIEER, Nashik, for guidance and support.